

Loans & Liquidity QA Test Plan

Every contract function a complete user interface must exercise across the Loans, LiquidityPool, CollateralManager, and ReferralDepositRouter contracts — organized as user flows a QA engineer can run top to bottom. Test IDs are stable; reference them in bug reports.

0 Environment & ground rules

Read this before running anything — it explains approvals and the deployment quirks that cause most integration bugs.

- **Chains.** BSC mainnet (56) and BSC testnet (97). Testnet was redeployed 2026-08-31.
- **Contracts.** Loans, LiquidityPool, and ReferralDepositRouter are the user-facing surface. CollateralManager is called internally by Loans, but it is the spender for collateral approvals and the source of supported-collateral reads. All are UUPS proxies — always call through the proxy address.
- **ABI caution.** The mainnet contracts have been upgraded many times. Use the ABIs supplied with this plan, never ABIs scraped from an explorer, and prefer the aggregate views (`getPoolStatus`, `getLiquidityStatus`, `getUserStatus`) over raw storage getters — some raw getters are stale on the deployed version.
- **Tokens.** LUSD (loan/stable token — what is borrowed, repaid, and what LP value is denominated in), LMLN (origination fee token), collateral assets from `CollateralManager.getSupportedAssets()` (e.g. WLEMX), and native BNB for gas.
- **Approval spenders.** Collateral token to CollateralManager · LMLN origination fee to LiquidityPool · LUSD loan payments to Loans · LP deposit tokens to LiquidityPool · referral deposits to ReferralDepositRouter. Our web app adds a ~10% buffer on the LMLN fee approval so `transferFrom` never comes up short.
- **Simulate first.** Run `eth_call` / gas estimation with the full ABI before every wallet prompt so custom errors are decoded and shown to the user, instead of a revert after gas is spent.
- **Loan status enum.** `loanStatus(loanId)` returns: 0 COMPLETED (paid, collateral withdrawn) · 1 UNLOCKED (paid, collateral withdrawable) · 2 DEFAULT · 3 ACTIVE · 4 LIQUIDATED.

1 Loan quoting & configuration — read-only

Everything the loan calculator screen must get right before a single transaction is sent.

ID	TEST	FUNCTIONS	EXPECTED
Q-01	Display global loan limits	loanConfig maxLoanAmount	Min/max amount and duration shown; UI blocks out-of-range input
Q-02	Show available borrowing liquidity	availablePrincipal getLiquidityStatus	Loan amount capped at available principal
Q-03	APR by duration	getInterestApr getAllInterestAprConfigs	APR updates as duration changes and matches the config table; format with aprDecimals
Q-04	Origination fee by LTV	getAllOriginationFeePcts	Fee tiers shown per LTV option; format with ltvDecimals
Q-05	Full quote	calculateLoanDetails	Interest, APR, origination fee, collateral required, cycle duration, and first payment match on-chain values exactly
Q-06	Collateral requirement	getRequiredCollateral getCollateralValue getSupportedAssets isAssetAllowed	Collateral amount and USD value correct per asset; only allowed assets offered

2 Loan creation

The full initiate flow: two approvals, the initiate call, and every way it can be rejected.

ID	TEST	FUNCTIONS	EXPECTED
C-01	Fresh-wallet approval flow	approve ×2	Collateral approved to CollateralManager, LMLN to LiquidityPool; steps skip when allowance is already sufficient
C-02	Happy path	initiateLoan(asset, duration, amount, ltv, originationPayer)	Returns loanId; loan appears via getAccountLoanIds; collateral and LMLN debited; LUSD credited
C-03	Verify stored terms	loans(loanId) loanStatus	Stored amount, duration, LTV, interest, and collateral match the quote; status = ACTIVE
C-04	Below min / above max amount	initiateLoan	Reverts; UI blocks before the wallet prompt
C-05	Unsupported duration or LTV	initiateLoan	Reverts (no APR config / fee tier); UI never offers these
C-06	Insufficient pool liquidity	initiateLoan	Reverts when amount exceeds availablePrincipal; clear message
C-07	Delegated fee payer	setOriginationDelegate initiateLoan(..., originationPayer)	Delegate's LMLN debited; a non-delegated payer reverts
C-08	Insufficient balances	collateral / LMLN	Each shortfall caught pre-prompt with a specific message
C-09	Paused contract	paused	Writes revert; UI shows a maintenance state

3 Active loans & payments

Loan list, cycle schedule, the default countdown, and every payment shape.

ID	TEST	FUNCTIONS	EXPECTED
P-01	Loan list with pagination	getAccountLoanIds(account, offset, limit)	All loans listed; pagination works past the limit
P-02	Cycle & schedule display	transpiredCycles remainingCycles fullCyclesAhead elapsedTimeInCycle remainingTimeInCycle totalNumberOfPayments	Brand-new loan shows 0/N transpired; counters advance with time
P-03	Default countdown	timeToDefault	Positive countdown on a healthy loan; shrinks when a cycle lapses unpaid
P-04	Minimum payment	loanPayment(loanId)	Amount due shown correctly (tiny loans may round to cents)
P-05	Make a payment	approve (LUSD to Loans) makeLoanPayment(loanId, amount)	paidAmount increments; remaining balance and cycle counters update
P-06	Multi-cycle prepayment	makeLoanPayment	fullCyclesAhead increases; timeToDefault extends
P-07	Final / balloon payment	makeLoanPayment	Status flips to UNLOCKED; balloon grace period respected
P-08	Payment reverts	makeLoanPayment	Zero amount, wrong loanId, someone else's loan — each reverts with a decoded error surfaced pre-prompt

4 Loan extension

Extending term costs a fresh origination fee — quote it like a new loan.

ID	TEST	FUNCTIONS	EXPECTED
E-01	Extend an active loan	extendLoan(loanId, extendTime, originationPayer)	Duration extends; a new LMLN origination fee is charged; new terms reflected in loans(loanId)
E-02	Extension quote	calculateLoanDetails / fee tiers	Fee for the extension shown before confirming
E-03	Extension limits	extendLoan	Reverts past maxLoanDuration or on a non-ACTIVE loan
E-04	Delegated extension fee	extendLoan	Same delegation rules as C-07 when originationPayer differs from the borrower

5 Payoff, collateral & default states

Terminal states. Defaults are processed by a keeper — the wallet only has to display them honestly.

ID	TEST	FUNCTIONS	EXPECTED
D-01	Withdraw collateral	<code>withdrawCollateral</code>	On an UNLOCKED loan: collateral returned in full; status becomes COMPLETED
D-02	Premature withdrawal	<code>withdrawCollateral</code>	Reverts on an ACTIVE loan
D-03	Defaulted loan display	<code>timeToDefault</code> (keeper: <code>processDefaults</code>)	Status becomes DEFAULT / LIQUIDATED; UI shows the terminal state and hides payment / withdraw actions
D-04	Historical loans	<code>loans(loanId)</code>	COMPLETED and LIQUIDATED loans render with final paid amounts

6 Liquidity discovery — read-only

The reads behind the liquidity dashboard and the deposit form's previews.

ID	TEST	FUNCTIONS	EXPECTED
LD-01	Supported deposit assets	<code>getSupportedAssets</code> <code>getAssetConfig</code>	Only active assets offered; per-asset config respected
LD-02	Lock tiers & multipliers	<code>getAssetLockTiers</code> <code>getMaxLockDuration</code>	Tier durations and interest multipliers displayed; disabled tiers hidden
LD-03	Pool overview	<code>getPoolStatus</code> <code>getShareValue</code> <code>getCurrentUtilization</code> <code>getLiquidityStatus</code> (Loans)	TVL, share value, utilization, and interest earned/distributed all render
LD-04	Deposit preview	<code>getDepositCredit</code> <code>getTokensForStableValue</code>	Credited stable value shown before deposit, including non-stable assets
LD-05	Minimums & caps	<code>minimumDepositValue</code> <code>minimumWithdrawalValue</code> <code>getAcceptableLiquidityAmount</code>	UI enforces limits before prompting

7 Deposits

Four deposit variants plus the referral router — each lands value differently.

ID	TEST	FUNCTIONS	EXPECTED
L-01	Stable-token deposit	approve (to LiquidityPool) deposit(token, amount, lockDuration, nonEarning)	Shares minted; getUserStatus principal increases by the credited value
L-02	Non-stable asset deposit	deposit	Credit matches getDepositCredit; token queued for swap (processSwaps is keeper-run)
L-03	Locked deposit, each tier	deposit getUserDepositEntries boostExpiriesOf boostPosition	Lock recorded; boost visible; earnings multiplier applied
L-04	No-lock and nonEarning variants	deposit(..., 0, ...) / nonEarning = true	No-lock earns the base rate; a nonEarning position earns nothing
L-05	Below minimum	deposit	Reverts under minimumDepositValue; blocked pre-prompt
L-06	Deposit for another account	depositFor(..., recipient)	Position lands on recipient, not the sender
L-07	Referral deposit	depositWithReferral tierRateFor cumulativeReferred allowedCommissionsList	Deposit credited to destination; referrer commission at tier rate; a disallowed commissions contract reverts
L-08	Paused pool	paused	Deposit UI disabled with clear messaging

8 Earnings

Claiming, compounding, the distribution cadence, and what happens when a lock boost expires.

ID	TEST	FUNCTIONS	EXPECTED
ER-01	Pending earnings display	previewPendingEarnings getUserStatus.pendingEarnings	Non-zero after interest accrues; the two reads agree
ER-02	Claim	claimEarnings	LUSD received; totalClaimedEarnings increments; pending resets
ER-03	Compound	compoundEarnings(lockDuration)	Earnings converted to new principal at the chosen lock; principal grows
ER-04	Distribution cadence	getPoolStatus.nextEarningsWith drawalTime earningsFrequency	UI shows when new earnings become available
ER-05	Lock expiry behavior	boostExpiriesOf expiryInfo	An expired boost stops multiplying; principal becomes unlocked in getUserStatus

9 Withdrawals — queue-based

Withdrawals are a two-step request-then-claim queue, not an instant transfer — the UI must make the wait legible.

ID	TEST	FUNCTIONS	EXPECTED
W-01	Unlocked vs locked principal	<code>getUserStatus</code>	Only <code>unlockedPrincipal</code> is withdrawable; UI separates the two
W-02	Request withdrawal	<code>requestWithdrawal(amount)</code>	Returns <code>requestId</code> ; appears in <code>getUserWithdrawalRequests</code> ; shares reduced
W-03	Queue visibility	<code>getWithdrawalQueueStatus</code> <code>getWithdrawalRequest</code>	Position in queue and funded state shown; the user understands the wait
W-04	Claim funded withdrawal	<code>claimWithdrawal(requestId)</code>	LUSD received once funded; claiming an unfunded request reverts
W-05	Limits	<code>minimumWithdrawalValue</code> <code>MAX_OPEN_REQUESTS_PER_ACCOUNT</code>	Below-minimum, too many open requests, or amount above unlocked — each reverts, blocked pre-prompt
W-06	Queue funding	<code>fundWithdrawalQueue</code>	Advances the queue when the pool has liquidity (verify caller permissions on the target deployment)

10 Account & cross-cutting

Behaviors that span every flow: pauses, price staleness, allowances, decimals, and multi-chain config.

ID	TEST	FUNCTIONS	EXPECTED
X-01	Full LP position summary	<code>getUserStatus</code> <code>getUserDepositEntries</code>	Principal, locked/unlocked split, pending and claimed earnings all correct across mixed deposits
X-02	Account transfer	<code>transferAccount(to)</code>	Entire LP position moves to <code>to</code> ; irreversible — require explicit double confirmation in the UI
X-03	Pause handling everywhere	<code>paused</code> on <code>Loans</code> , <code>LiquidityPool</code> , <code>Router</code>	All writes disabled with clear messaging; reads keep working
X-04	Price feed staleness	any pricing-dependent call	The real reason (e.g. <code>PriceStale</code>) is surfaced pre-prompt, not a doomed transaction
X-05	Allowance edge cases	<code>allowance approve</code>	Approval steps reappear only when needed; LMLN approval covers the origination fee (buffer recommended)
X-06	Decimals & formatting	<code>aprDecimals</code> <code>ltvDecimals</code> <code>token decimals</code>	No off-by-10 ⁿ display bugs across LUSD, LMLN, collateral, and BNB
X-07	Multi-chain config	<code>chain 56 vs 97</code>	Correct addresses per chain; graceful behavior on unsupported chains
X-08	Concurrent state changes	<code>two wallets / refresh mid-flow</code>	UI refetches after each confirmed transaction; no stale quotes submitted

Test IDs are stable — cite them in bug reports. Contract ABIs accompany this plan (`Loans`, `LiquidityPool`, `CollateralManager`, `ReferralDepositRouter`). `Admin`, `keeper`, and `upgrade` functions (`role grants`, `config setters`, `processDefaults`, `processSwaps`, `upgradeTo...`) are intentionally out of scope for the wallet UI.